

Lab #7

Group number : 5

Group members : Aditya Ray - 1233750715

Krutyanjay Shinde - 1234108189

PRE LAB

1. Can we use the gyroscope to make the robot navigate the curves?

- Technically yes, we can use the gyroscope, but we shouldn't.
- To use a gyroscope for the curves, we would need to know exactly how sharp the curve is in advance. Since the gyroscope doesn't actually 'see' the line, if the robot slips or the track is slightly different than expected, the robot will drive off the track without knowing it.
- It is much better to use the IR line sensors because they constantly detect exactly where the line is. This allows the robot to adjust its steering immediately if it starts to drift away, rather than just guessing the path.

2. Can we use the line sensors on the edges, i.e., sensors 1 and 5, to detect angles and curves?

- No, using only sensors 1 and 5 is not enough.
- First, line sensors only detect 'black' or 'white.' They do not measure angles directly.
- Second, if we only use the edge sensors, we have a blind spot. If both sensors 1 and 5 see white, we don't know if the robot is perfectly centered (line is under the middle sensor) or if the robot is completely off the track. We need to read all sensors (1 through 5) to know exactly where the line is relative to the center of the robot. This pattern allows us to calculate how much to steer to get back on track smoothly.

POST LAB

1. What were your takeaways from the lab? What did you learn during the lab? Did any results in the lab surprise you?

- In this lab I learned how much the control logic actually matters for line following.
- At first I tried a very naive on/off approach: if a sensor went past a threshold and looked "out of line," I just turned the robot in the opposite direction. It did kind of work, but it caused a lot of jittering, even on straight sections, and it got worse as I increased the speed. That was surprising to me—I expected that simple logic to be good enough.
- Then I switched to using a weighted average of the sensor values to estimate the line position and an error-proportional control for the motor speeds. That made the robot much smoother and better at handling curves and sharper turns.

Overall, I learned why using the “how far off” information and having proper control logic is important instead of just simple on/off decisions.

2. What method or logic did you use to tackle curves and sharp angles during the experiment?

- We used a weighted-average method over the five line sensors to estimate a continuous line position between 0 and 4000, instead of just picking the max sensor. The controller then computes an error relative to the center (2000) and applies proportional steering: the left and right wheel speeds are set to $v \pm K_p \cdot \text{error}$. On curves and sharp angles this increases the speed difference between the wheels (stronger turning) and, if desired, we reduce overall speed for large errors so the robot can negotiate tighter bends without losing the line.

3. Justify or explain the logic behind choosing that particular method.

- We first tried a naive approach: we read the sensor values and compared each one to a fixed threshold. If a sensor went “out of line” (crossed the threshold), we turned the robot in the opposite direction of that sensor. For example, if a left sensor detected that it was off the line, we steered to the right, and vice versa.
- This worked in the sense that the robot followed the line, but it caused a lot of jittery motion even on straight segments. As soon as one side detected it was off the line, the robot turned aggressively to compensate, which often pushed the line under the opposite sensors. Then those sensors would trigger and compensate back the other way. This back-and-forth switching produced visible oscillations, and the effect became worse at higher speeds.
- Sharp turns also behaved poorly. For example, on a left corner the left sensors would first see the line and command a strong turn to the right. Only once the center (and then right) sensors lost the line would the controller finally react and turn left, so the robot tended to take wide, imprecise turns.
- To solve this, we switched to using a weighted average of the sensor readings to estimate a continuous line position, and then computed an error relative to the center of the robot. We used this error in a proportional controller for the motor speeds. This “error-proportional” logic gives small corrections when the robot is close to the center of the line and larger corrections when it is farther away, which significantly reduced jitter and allowed the robot to handle curves and sharp angles more smoothly.